

## **Chapter 3 Review - Questions and Answers**

### **3.1 State whether the following statements are true or false.**

- (a) True
- (b) False
- (c) False
- (d) True
- (e) True
- (f) True
- (g) False
- (h) False
- (i) False
- (j) False
- (k) True
- (l) True

### **3.2 Fill in the blanks with appropriate words.**

- (a) const
- (b) 65535
- (c) external
- (d) typedef
- (e) UL
- (f) struct and union
- (g) int

### **3.3 What are trigraph characters?**

A set of three-character sequences that represent a single character in C.

### **3.4 Describe the four basic data types.**

int, float, char, double.

### **3.5 What is an unsigned integer constant?**

A constant without a sign that can store only positive values.

### **3.6 Characteristics and purpose of escape sequence characters.**

Special characters preceded by a backslash, e.g., '\n', '\t'.

### **3.7 What is a variable and its value?**

A variable stores data, and its value is the data it holds.

### **3.8 Difference between variables and symbolic names.**

Variables store values; symbolic names (constants) cannot change.

### **3.9 Difference between variable declaration and symbolic name definition.**

A variable is a storage location; a symbolic name (macro) is a constant.

### **3.10 Qualifiers for int.**

signed, unsigned, short, long.

### **3.11 Using DPR for double precision values.**

typedef double DPR;

### **3.12 Enumeration variables.**

Used to define named integer constants using enum.

### **3.13 Purpose of const and volatile.**

const prevents modification; volatile tells the compiler the value may change externally.

### **3.14 Improving accuracy for small/large numbers.**

Use double/long double and scientific notation.

### **3.15 Identify invalid constants.**

Invalid: Minimum, 5 x 1.5 (invalid multiplication).

### **3.16 Identify invalid variable names.**

Invalid: First.name, 99999, "15.75", n1+n2, n\$.

### **3.17 Value of x after execution.**

x = 107.

### **3.18 Explanation of enumerated types and typedef.**

- Enum: User-defined data type with named constants.
- Typedef: Creates an alias for a data type.

### **3.19 Differences:**

(a) Global vs. Local: Global accessible everywhere, local only within scope.

(b) Initialization vs. Assignment: Initialization at declaration, assignment later.

(c) Automatic vs. Static: Automatic exists within scope, static retains value across function calls.